

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
ITMO University

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ 6

По дисциплине Web-программирование

Тема работы Работа с сокетами

Обучающийся Боженко Мария Александровна

Факультет инфокоммуникационных технологий

Группа K3321

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся	<u>21.04.2025</u> (дата)	<u> </u> (подпись)	<u>Боженко М.А.</u> (Ф.И.О.)
Руководитель	<u> </u> (дата)	<u> </u> (подпись)	<u>Марченко Е.В.</u> (Ф.И.О.)

Санкт-Петербург
2025

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 Обмен сообщениями между клиентом и сервером	4
2 Ввод параметров на стороне клиента	6
3 Вывод HTML-страницы	10
4 Многопользовательский чат	12
ЗАКЛЮЧЕНИЕ	16

ВВЕДЕНИЕ

Цель на данную лабораторную работу: Овладеть практическими навыками и умениями реализации web-серверов и использования сокетов.

Задачи, поставленные в данной лабораторной работе:

1. Реализовать клиентскую и серверную часть приложения: осуществить взаимодействие клиента и сервера двумя способами
2. Реализовать серверную часть приложения: при подключении клиента, он получает сообщение содержащее html-страницу
3. Реализовать двухпользовательский или многопользовательский чат

1 Обмен сообщениями между клиентом и сервером

В данной части лабораторной требовалось реализовать клиентскую и серверную части приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отразиться у клиента.

Ниже представлены реализации сервера и клиента на python:

```
1  import socket
2
3
4  def start_server():
5
6      server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7      server_socket.bind(('localhost', 12345))
8      server_socket.listen(1)
9      print("Сервер запущен. Ожидание подключения...")
10
11     client_socket, addr = server_socket.accept()
12     print(f"Подключено к {addr}")
13
14     message = client_socket.recv(1024).decode('utf-8')
15     print(f"Получено сообщение от клиента: {message}")
16
17     response = "Hello, client!"
18     client_socket.send(response.encode('utf-8'))
19
20     client_socket.close()
21     server_socket.close()
22
23
24  if __name__ == "__main__":
25     start_server()
```

Рисунок 1.1 — Реализация сервера

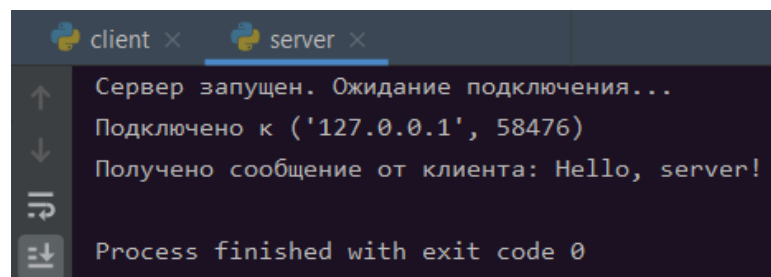
```

1  ► import socket
2
3
4  def start_client():
5
6      client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
7
8      client_socket.connect(('localhost', 12345))
9
10     # message = "Hello, server! :)"
11     message = input("Введите сообщение для сервера: ")
12     client_socket.send(message.encode('utf-8'))
13
14     response = client_socket.recv(1024).decode('utf-8')
15     print(f"Получен ответ от сервера: {response}")
16
17     client_socket.close()
18
19
20 if __name__ == "__main__":
21     start_client()
22

```

Рисунок 1.2 — Реализация клиента

Ниже представлены результаты работы кода на стороне сервера и на стороне клиента:

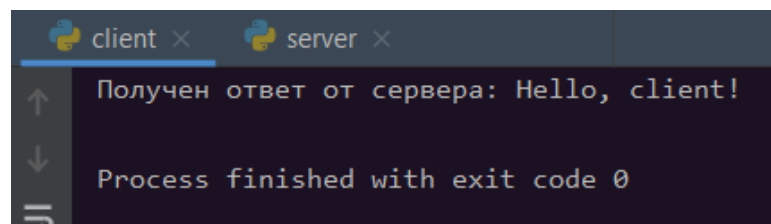


```

client x server x
↑ Сервер запущен. Ожидание подключения...
↓ Подключено к ('127.0.0.1', 58476)
↻ Получено сообщение от клиента: Hello, server!
⇓ Process finished with exit code 0

```

Рисунок 1.3 — Результат на стороне сервера



```

client x server x
↑ Получен ответ от сервера: Hello, client!
↓
↻
⇓ Process finished with exit code 0

```

Рисунок 1.4 — Результат на стороне клиента

2 Ввод параметров на стороне клиента

В данном задании требовалось также реализовать клиентскую и серверную части приложения. Клиент запрашивает у сервера выполнение математической операции, параметры которой вводятся с клавиатуры. В соответствии с вариантом сервер решал задачу нахождения площади параллелограмма. Сервер обрабатывает полученные данные и возвращает результат клиенту.

Код для сервера и клиента аналогичен коду из прошлого задания. Для разнообразия задачи, были добавлены варианты решения задачи.

Ниже представлен код для сервера и клиента:

```
import socket

def start_server():

    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind(('localhost', 12345))
    server_socket.listen(1)
    print("Сервер запущен. Ожидание подключения...")

    client_socket, addr = server_socket.accept()
    print(f"Подключено к {addr}")

    request = "Уважаемый пользователь, выбери формулу: 1 - через высоту и основание; 2 - через синус"
    client_socket.send(request.encode('utf-8'))

    message = client_socket.recv(1024).decode('utf-8')
    print(f"Получено сообщение от клиента: {message}")

    if message == "1":
        request = "Уважаемый пользователь, введи основание и высоту ЧЕРЕЗ ПРОБЕЛ"
        client_socket.send(request.encode('utf-8'))

        message = client_socket.recv(1024).decode('utf-8')
        print(f"Получено сообщение от клиента: {message}")
```

Рисунок 2.1 — Код сервера (часть 1)

```

    a, h = message.split()
    s = int(a) * int(h)

    response = f"Площадь данного параллелограмма равна {s}"
    client_socket.send(response.encode('utf-8'))
else:
    request = "Уважаемый пользователь, введи синус и два основания ЧЕРЕЗ ПРОБЕЛ"
    client_socket.send(request.encode('utf-8'))

    message = client_socket.recv(1024).decode('utf-8')
    print(f"Получено сообщение от клиента: \033[30;45m{message}\033[0m")

    sin, a, b = message.split()
    s = float(sin) * int(a) * int(b)

    response = f"Площадь данного параллелограмма равна {s}"
    client_socket.send(response.encode('utf-8'))

client_socket.close()
server_socket.close()

if __name__ == "__main__":
    start_server()

```

Рисунок 2.2 — Код сервера (часть 2)

```

import socket

def start_client():

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    client_socket.connect(('localhost', 12345))

    request = client_socket.recv(1024).decode('utf-8')
    print(f"Получено сообщение от сервера: \033[30;44m{request}\033[0m")

    message = input("Введите сообщение для сервера: ")
    client_socket.send(message.encode('utf-8'))

    request = client_socket.recv(1024).decode('utf-8')
    print(f"Получено сообщение от сервера: \033[30;44m{request}\033[0m")

    message = input("Введите сообщение для сервера: ")
    client_socket.send(message.encode('utf-8'))

    response = client_socket.recv(1024).decode('utf-8')
    print(f"Получен ответ от сервера: \033[30;44m{response}\033[0m")

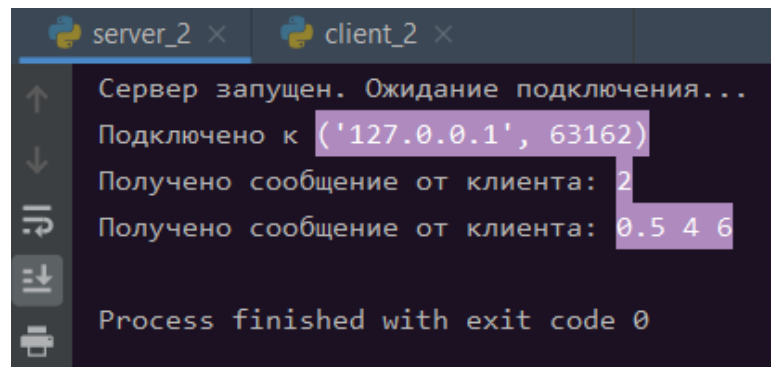
    client_socket.close()

if __name__ == "__main__":
    start_client()

```

Рисунок 2.3 — Код клиента

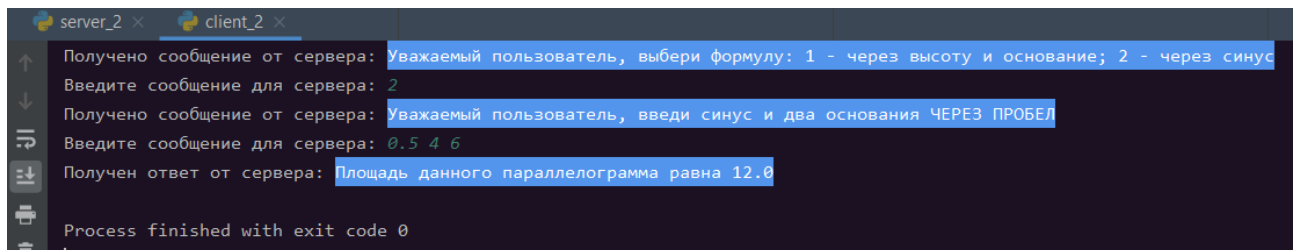
Ниже представлен результат работы программы на сторонах сервера и клиента:



The screenshot shows a terminal window with two tabs: 'server_2' and 'client_2'. The 'server_2' tab is active. The output text is as follows:

```
Сервер запущен. Ожидание подключения...  
Подключено к ('127.0.0.1', 63162)  
Получено сообщение от клиента: 2  
Получено сообщение от клиента: 0.5 4 6  
  
Process finished with exit code 0
```

Рисунок 2.4 — Результат на стороне сервера



The screenshot shows a terminal window with two tabs: 'server_2' and 'client_2'. The 'client_2' tab is active. The output text is as follows:

```
Получено сообщение от сервера: Уважаемый пользователь, выбери формулу: 1 - через высоту и основание; 2 - через синус  
Введите сообщение для сервера: 2  
Получено сообщение от сервера: Уважаемый пользователь, введи синус и два основания ЧЕРЕЗ ПРОБЕЛ  
Введите сообщение для сервера: 0.5 4 6  
Получен ответ от сервера: Площадь данного параллелограмма равна 12.0  
  
Process finished with exit code 0
```

Рисунок 2.5 — Результат на стороне клиента

3 Вывод HTML-страницы

В данном задании требовалось реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащее html-страницу, которую сервер подгружает из файла index.html

Ниже представлен код для сервера:

```
def start_http_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    server_socket.bind(('localhost', 8080))

    server_socket.listen(1)
    print("HTTP сервер запущен. Ожидание подключения...")

    while True:
        client_socket, addr = server_socket.accept()
        print(f"Подключено к {addr}")

        request = client_socket.recv(1024).decode('utf-8')
        print(f"Получени запрос:\n {request}")

        try:
            with open('index.html', 'r') as file:
                response_body = file.read()
            response_status = 'HTTP/1.1 200 OK\r\n'
        except FileNotFoundError:
            response_body = '<h1>404 Not Found :(</h1>'
            response_status = 'HTTP/1.1 404 Not Found\r\n'

        response_headers = 'Content-Type: text/html; charset=utf-8\r\n'
        response_headers += 'Content-Length: {}'.format(len(response_body))
        response_headers += '\r\n'

        client_socket.sendall(response_status.encode('utf-8'))
        client_socket.sendall(response_headers.encode('utf-8'))
        client_socket.sendall(response_body.encode('utf-8'))

        client_socket.close()
```

Рисунок 3.1 — Код для сервера

Результат в браузере при переходе на `http://localhost:8080` представлен ниже:

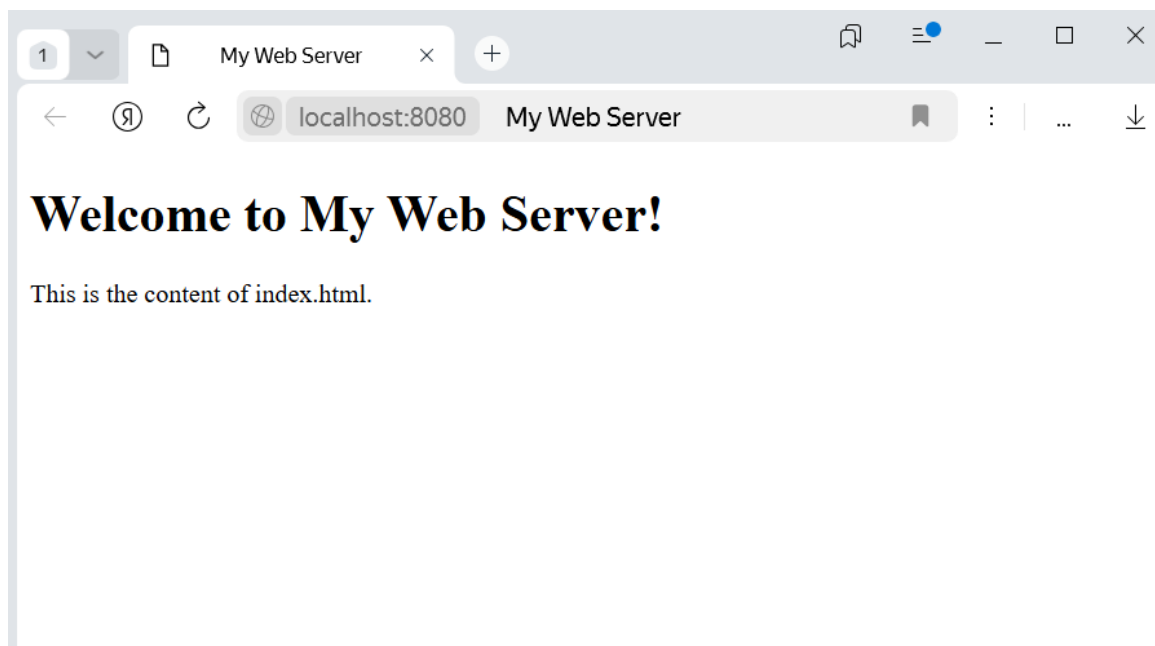


Рисунок 3.2 — Результат в браузере

Если по какой-то причине у сервера пропал доступ к странице `index.html` и она не была найдена, результат будет следующим:

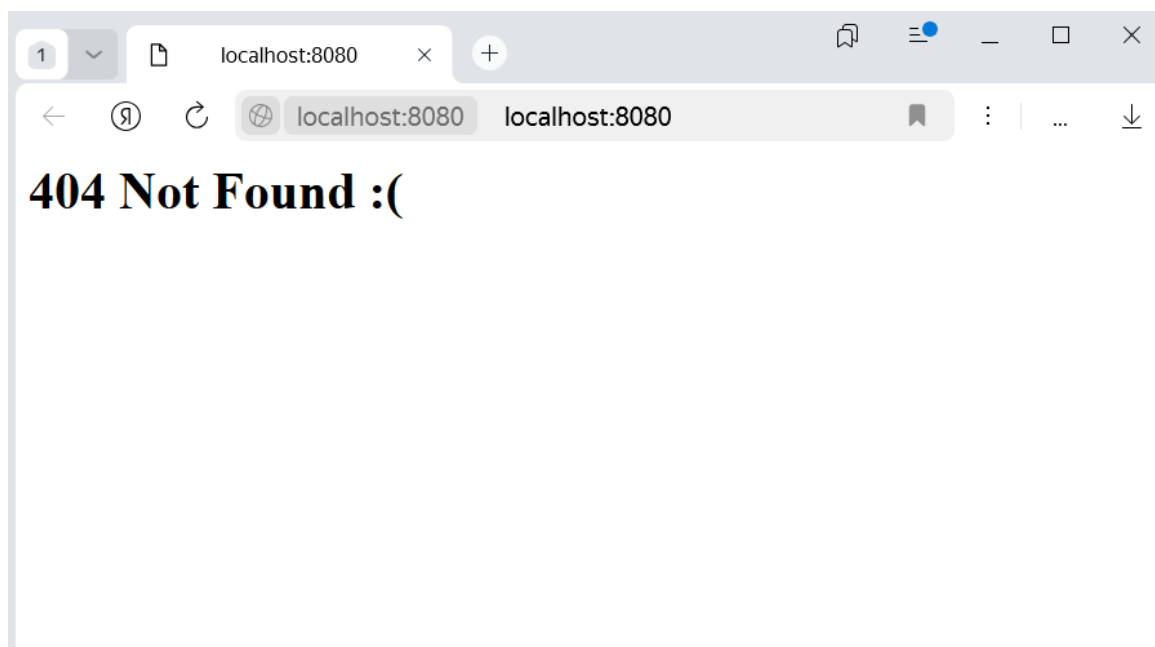


Рисунок 3.3 — Результат в браузере при ошибке

4 Многопользовательский чат

В последнем задании лабораторной требовалось реализовать двухпользовательский или многопользовательский чат.

Ниже представлен код для сервера:

```
import socket
import threading

HOST = '127.0.0.1'
PORT = 12345

clients = []

def handle_client(client_socket, addr):
    print(f"[NEW CONNECTION] {addr} connected.")

    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            if message:
                print(f"[{addr}] {message}")
                broadcast(message, client_socket)
            else:
                remove(client_socket)
                break
        except:
            continue
```

Рисунок 4.1 — Код сервера (часть 1)

```

def broadcast(message, client_socket):
    for client in clients:
        if client != client_socket:
            try:
                client.send(message.encode('utf-8'))
            except:
                remove(client)

def remove(client_socket):
    if client_socket in clients:
        clients.remove(client_socket)

def start():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((HOST, PORT))
    server_socket.listen(5)
    print("[SERVER STARTED] Waiting for connections...")

    while True:
        client_socket, addr = server_socket.accept()
        clients.append(client_socket)
        thread = threading.Thread(target=handle_client, args=(client_socket, addr))
        thread.start()

if __name__ == "__main__":
    start()

```

Рисунок 4.2 — Код сервера (часть 2)

Код для клиента представлен ниже:

```

import socket
import threading

HOST = '127.0.0.1'
PORT = 12345

def receive_messages(client_socket):
    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            if message:
                print(message)
            else:
                break
        except:
            print("An error occurred!")
            client_socket.close()
            break

```

Рисунок 4.3 — Код клиента (часть 1)

```

def start_client():
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client_socket.connect((HOST, PORT))

    thread = threading.Thread(target=receive_messages, args=(client_socket,))
    thread.start()

    while True:
        message = input()
        if message.lower() == "exit":
            break
        client_socket.send(message.encode('utf-8'))

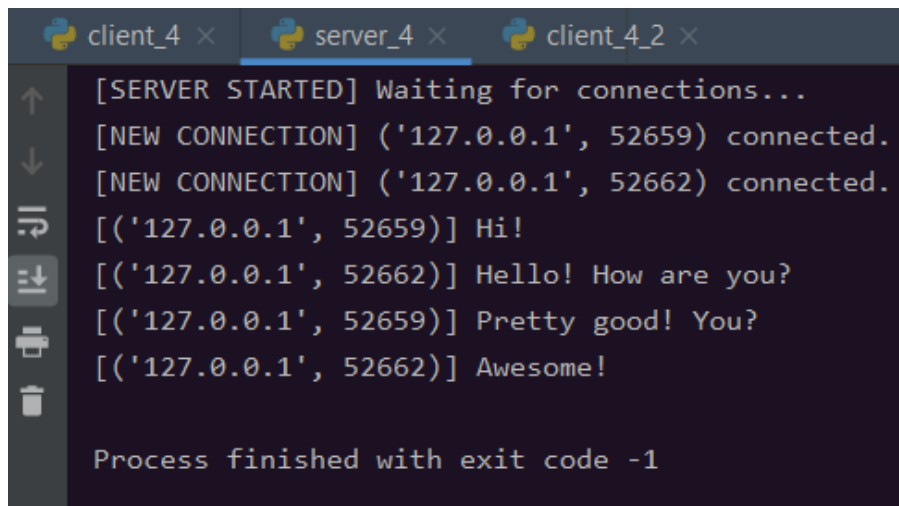
    client_socket.close()

if __name__ == "__main__":
    start_client()

```

Рисунок 4.4 — Код клиента (часть 2)

Ниже представлен результат со стороны сервера и двух пользователей, обменивающихся сообщениями.

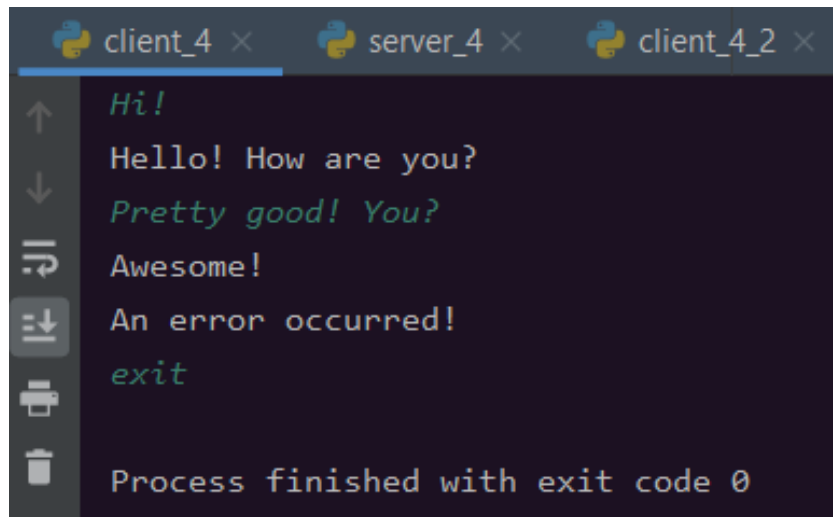


The screenshot shows a terminal window with three tabs: 'client_4', 'server_4', and 'client_4_2'. The 'server_4' tab is active. The output shows the server starting, waiting for connections, and then receiving two connections from '127.0.0.1'. The first connection (52659) says 'Hi!'. The second connection (52662) says 'Hello! How are you?'. The first connection then responds 'Pretty good! You?'. The second connection responds 'Awesome!'. Finally, the process finishes with exit code -1.

```
[SERVER STARTED] Waiting for connections...
[NEW CONNECTION] ('127.0.0.1', 52659) connected.
[NEW CONNECTION] ('127.0.0.1', 52662) connected.
[('127.0.0.1', 52659)] Hi!
[('127.0.0.1', 52662)] Hello! How are you?
[('127.0.0.1', 52659)] Pretty good! You?
[('127.0.0.1', 52662)] Awesome!

Process finished with exit code -1
```

Рисунок 4.5 — Результат на стороне сервера

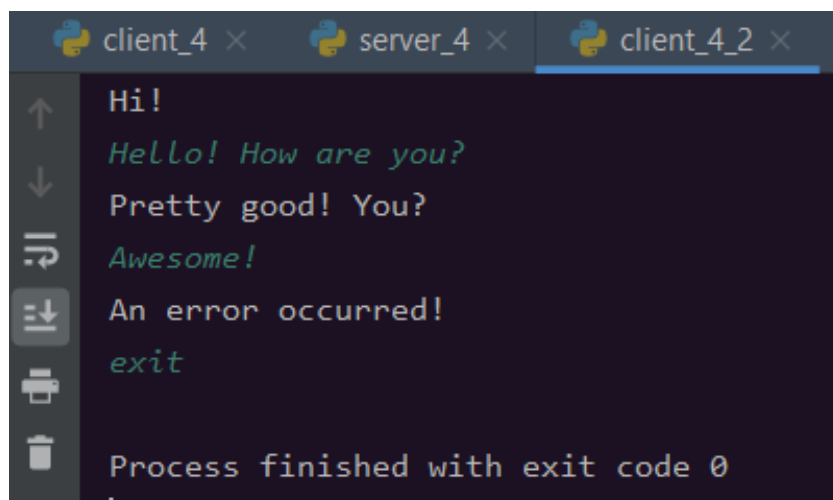


The screenshot shows a terminal window with three tabs: 'client_4', 'server_4', and 'client_4_2'. The 'client_4' tab is active. The output shows the client sending 'Hi!', 'Hello! How are you?', 'Pretty good! You?', and 'Awesome!'. Then, an error occurs, and the client sends 'exit'. The process finishes with exit code 0.

```
Hi!
Hello! How are you?
Pretty good! You?
Awesome!
An error occurred!
exit

Process finished with exit code 0
```

Рисунок 4.6 — Результат на стороне клиента 1



The screenshot shows a terminal window with three tabs: 'client_4', 'server_4', and 'client_4_2'. The 'client_4_2' tab is active. The output shows the client sending 'Hi!', 'Hello! How are you?', 'Pretty good! You?', and 'Awesome!'. Then, an error occurs, and the client sends 'exit'. The process finishes with exit code 0.

```
Hi!
Hello! How are you?
Pretty good! You?
Awesome!
An error occurred!
exit

Process finished with exit code 0
```

Рисунок 4.7 — Результат на стороне клиента 2

ЗАКЛЮЧЕНИЕ

В результате выполнения данной лабораторной работы были выполнены все поставленные задачи. Было проведено ознакомление с сокетами и применены практические навыки в работе с ними.